



DICTIONARY ABSTRACT DATA STRUCTURE HASH FUNCTIONS

MARZIEH ESKANDARI
NJIT

DICTIONARY

- We assign **keys to elements** and then use those keys later to look up or remove elements: (k,e)
- This functionality defines a data structure known as a **dictionary** or **map**.
- This structure supports methods for the **insertion, removal**, and **searching** of values in terms of keys associated with those values.



Keys are like labels assigned to file folders

DEFINITION: MAP

- A **map** stores a collection of **key-value pairs**, (k, v) , which we call **items**, where **k** is a **key** and **v** is a **value** that is associated with that key.
- For example, in a map storing student records (such as the student's name, address, and course grades), the key might be the student's ID number.
- We assume here that keys are **unique** then the key associated with an object can be viewed as an “**address**” for that object in memory.

DEFINITION: MULTIMAP

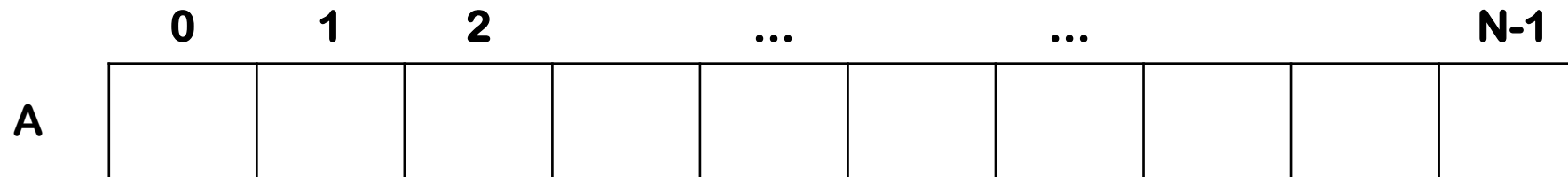
- A map that allows for multiple values for the same key is called a *multimap*.

METHODS

- As *map* data structure, M , supports the following fundamental methods:
 1. **get(k):** If M contains an item with key equal to k , then **return the value** of such an item; else return a special element NULL.
 2. **put(k, v):** **Insert an item** with key k and value v ; if there is already an item with key k , then replace its value with v .
 3. **remove(k):** **Remove** from M an **item with key** equal to k , and return this item. If M has no such item, then return the special element NULL.

IMPLEMENTING A MAP

- In some cases, the universe of **keys** that will be used for a map is the set of integers in the **range** $[0, N - 1]$, for a reasonably small value of N .
- For implementing a map: we can use an **array**, A , of size N .

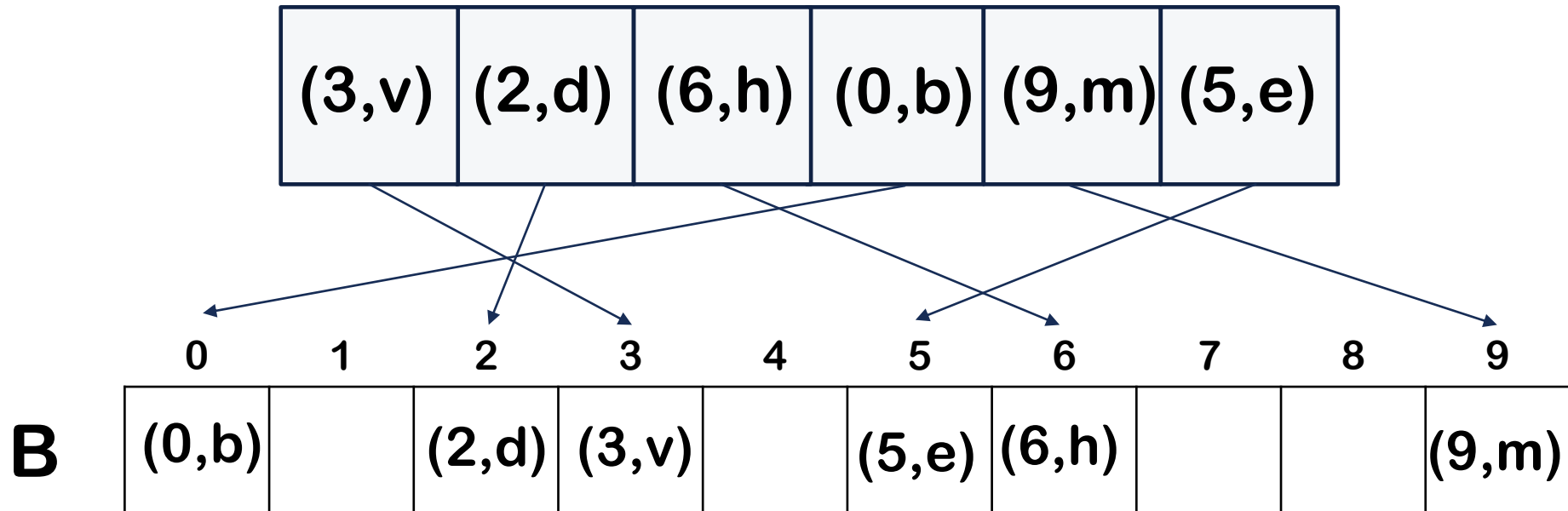


LOOKUP TABLES

- We can use A to store any key-value pair (k, v) by placing that pair in the cell $A[k]$. We refer to this implementation as a *lookup table*.

	0	1	2	...	k	...			N-1
A					(k,v)				

EXAMPLE



LOOKUP TABLES

- We perform the map operations as follows:
 1. To perform a `put(k, v)` operation, we assign (k, v) to $A[k]$.
 2. To perform a `get(k)` operation, we return $A[k]$.
 3. To perform a `remove(k)` operation, we return $A[k]$ and then we assign the NULL item to $A[k]$.

$O(1)$

GENERALIZATION

1. Keys are integers BUT NOT in a range $[0, N - 1]$ for a reasonable small N : Define a function to bring them into the range
2. Keys are NOT single integers, they can be viewed as a d -tuple, of the form $(x_1, x_2, \dots, x_d)$ in which $x_i \in [0, M - 1]$, for a reasonable small M : Define a function to map $(x_1, x_2, \dots, x_d)$ to a single integer in the range

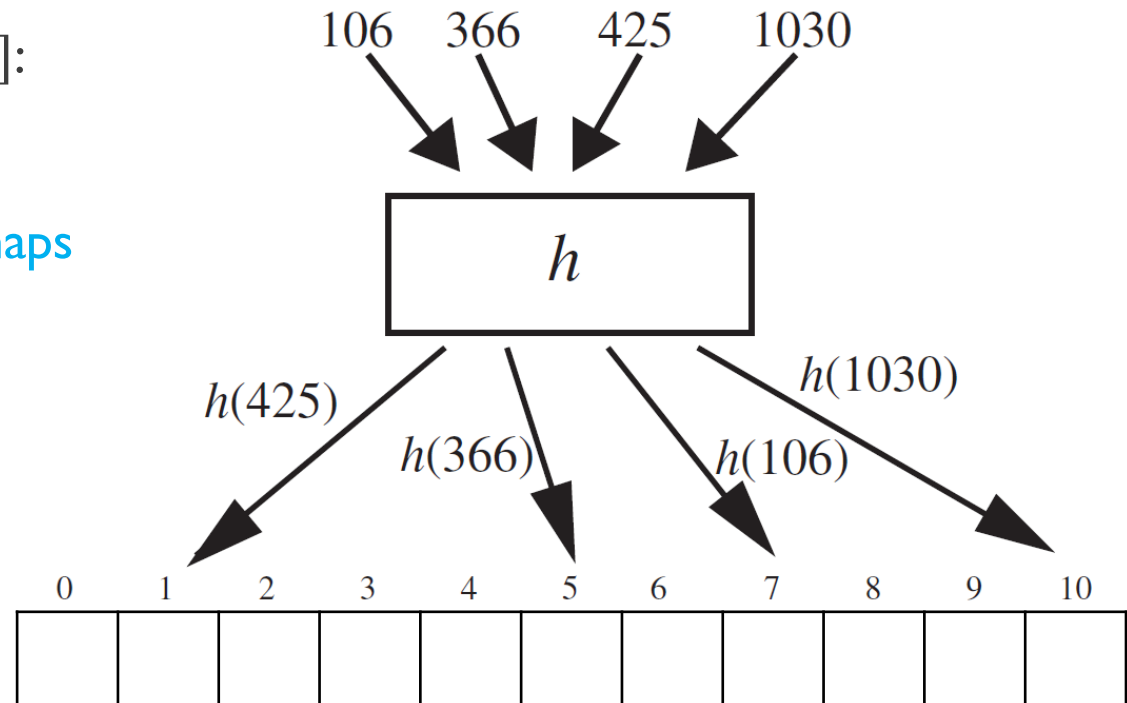
EXAMPLE

1. $[2, 5, 12543, 3487782, 445, 34, 1, 56, 94898]$ belong to the range $[0, 3487782]$
2. list of words: $Hello = (H, e, l, l, o) = (72, 101, 108, 108, 108, 111)$ range of components is: $[0, 255]$
3. List of IP addresses: $192.168.1.1 = (192, 168, 1, 1)$ range of components is: $[0, 255]$

HASH FUNCTION

- If keys are NOT integers in the range $[0, N - 1]$:

We use a function, h , called a **hash function**, that **maps** each **key** k to an **integer in the range** $[0, N - 1]$.

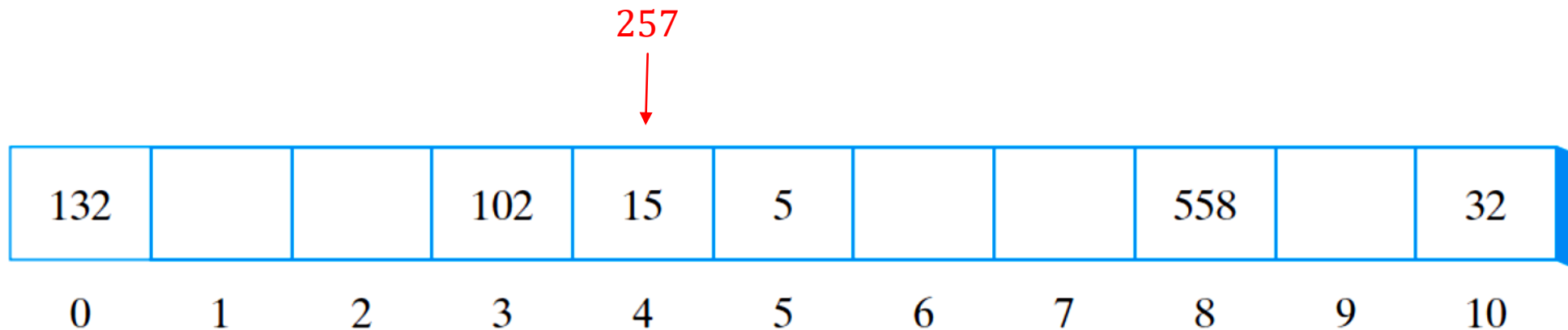


HASH FUNCTION: COLLISION

- One complication that we have to deal with when using this approach, is that it is possible that the **hash function**, h , could **map two distinct keys to the same location** in A .
- We could have $h(k) = h(l)$, for $k \neq l$, for two keys k and l in our map.
- We refer to **such a pair of keys with the same hash value, j** , as causing a **collision at j** .

HASH FUNCTIONS: EXAMPLE

- Suppose that we want to store 257. If $h(257) = 4$, then 257 should be stored at location 4;
- This position is already occupied. In this case we say that a **collision** has occurred.



HASH FUNCTIONS ON INTEGER KEYS

- In the case that we can view the key k as a single integer, there are a number of different kinds of hash functions, such as:
 1. Modular Division
 2. Random Linear Functions
 3. Random Polynomial Functions

MODULAR DIVISION

- In the case that we can view the key k as a single integer, perhaps the simplest hash function is to use modular division to compress k into the range $[0, N - 1]$ as follows:

$$h(k) = k \bmod N.$$

$$h(k) = k \bmod 11$$

132			102	15	5			558		32
0	1	2	3	4	5	6	7	8	9	10

RANDOM LINEAR FUNCTIONS

- A more sophisticated compression function, is a *random linear hash function*.
- In this case, we define a *random linear hash function*, h , for an integer key, k , as

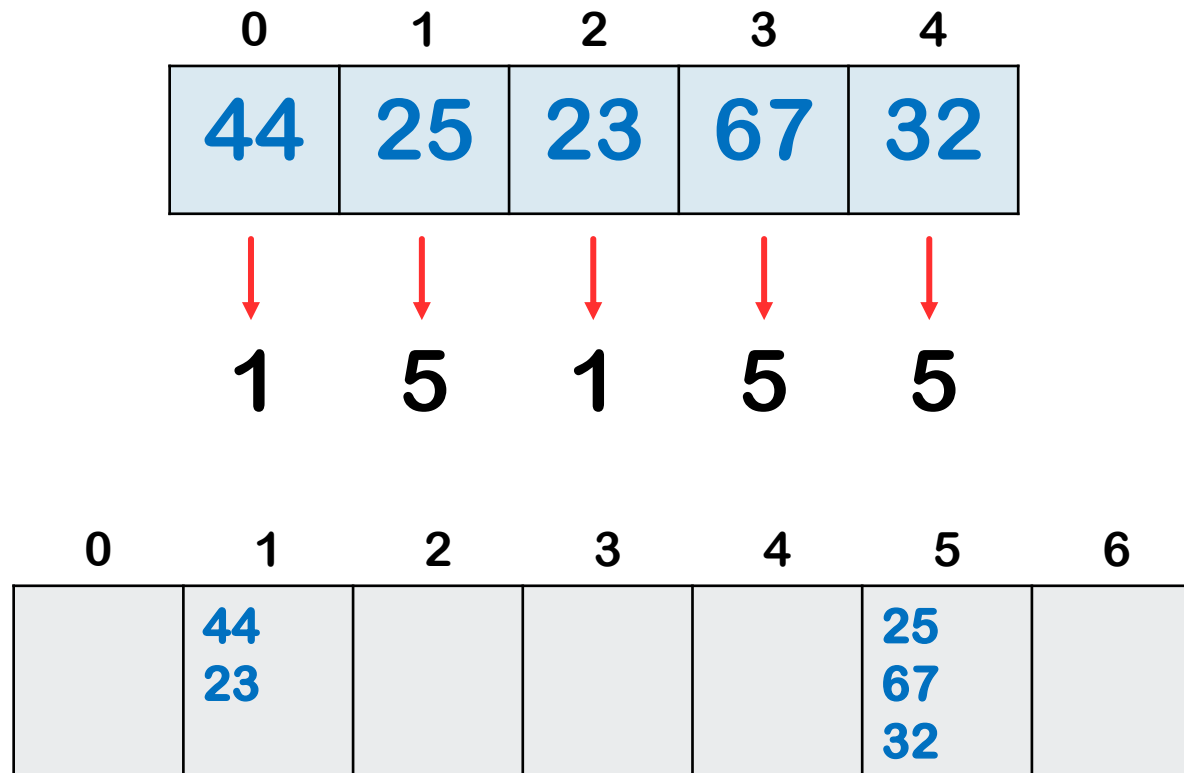
$$h(k) = (ak + b) \bmod N$$

where N is a prime number, and $0 < a < N$ and $0 \leq b < N$ are independent random integers.

EXAMPLE

$$h(k) = (2k + 4) \bmod 7$$

$$0 < 2 < 7 \text{ and } 0 \leq 4 < 7$$



RANDOM POLYNOMIAL FUNCTIONS

- We may use a random polynomial hash function, h , which is defined as

$$h(k) = a_d + k a_{d-1} + k^2 a_{d-2} + \cdots + k^{d-1} a_1 \bmod N$$

By Horner's rule

$$h(k) = a_d + k \left(a_{d-1} + k \left(a_{d-2} + \cdots + k \left(a_3 + k \left(a_2 + k a_1 \right) \right) \right) \right) \bmod N$$

where N is prime and the a_i 's are independent uniformly random integers in the range $[0, N - 1]$, such that at least one of the coefficients in $\{a_1, a_2, \dots, a_{d-1}\}$ is nonzero.

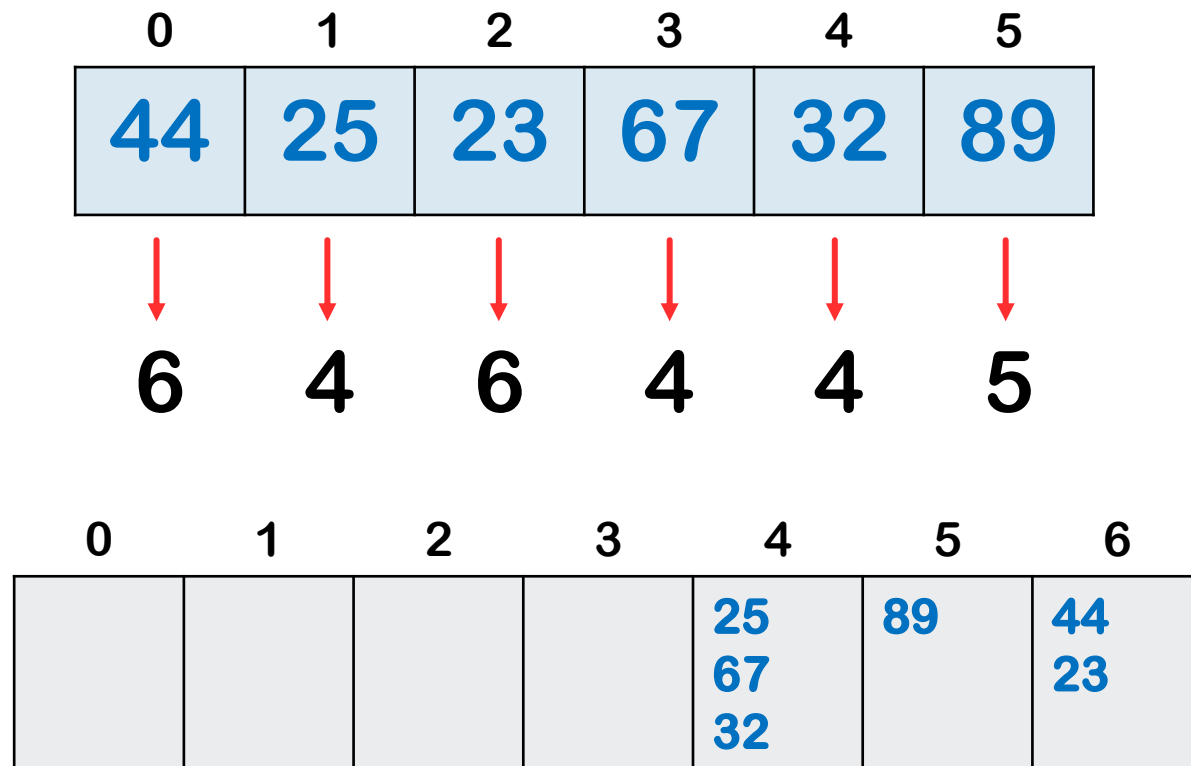
EXAMPLE

$N=7$

$d = 3: \{a_1, a_2, a_3\} = \{3, 2, 4\}$

$$h(k) = (3k^2 + 2k + 4) \bmod 7$$

$$h(k) = (4 + k(2 + 3k)) \bmod 7$$



VIEWING KEYS AS TUPLES

- If keys are not single integers, they can be viewed as a d -tuple, of the form

$$k = (x_1, x_2, \dots, x_d)$$

where each x_i is an integer in range $[0, M - 1]$, for some M and d .

VIEWING KEYS AS TUPLES: EXAMPLE

- For example, if our keys are **character strings** (such as words taken from web pages) then it is probably most natural to view such keys as **tuples**:

$$(H, e, l, l, o) = (72, 101, 108, 108, 108, 111)$$

- If our keys are **fixed length IP addresses** of destination domains on the Internet, then it is probably most natural to view them as **single integers**:

$$192.168.1.1 = (192, 168, 1, 1)$$

HASH FUNCTIONS ON TUPLES

- In the case when each of our keys, k , is a d -tuple, of the form

$$k = (x_1, x_2, \dots, x_d)$$

There are a number of different kinds of hash functions, such as:

1. Summing Components
2. Polynomial-Evaluation
3. Tabulation-based

HASH FUNCTION: SUMMING COMPONENTS

- In the case when each of our keys, k , is a d -tuple, of the form

$$k = (x_1, x_2, \dots, x_d)$$

one possible hash function we could use is **to sum up the different components** in each key. That is, we could compute $h(k)$ as

$$h(k) = \sum_{i=1}^d x_i$$

EXAMPLE

0	1	2	3	4	5	6	7	8	9
(2,9,1)	(3,8,8)	(4,3,0)	(2,2,4)	(5,3,3)	(1,8,0)	(4,1,5)	(9,1,6)	(2,2,1)	(4,6,4)



12 19 7 8 11 9 10 16 5 14

$$h(k) = \sum_{i=1}^3 x_i$$



12 6 7 8 11 9 10 3 5 1

$$h(k) \equiv \sum_{i=1}^3 x_i \text{ mod } 13$$

0	1	2	3	4	5	6	7	8	9	10	11	12
	(4,6,4)		(9,1,6)		(2,2,1)	(3,8,8)	(4,3,0)	(2,2,4)	(1,8,0)	(4,1,5)	(5,3,3)	(2,9,1)

XORING COMPONENTS

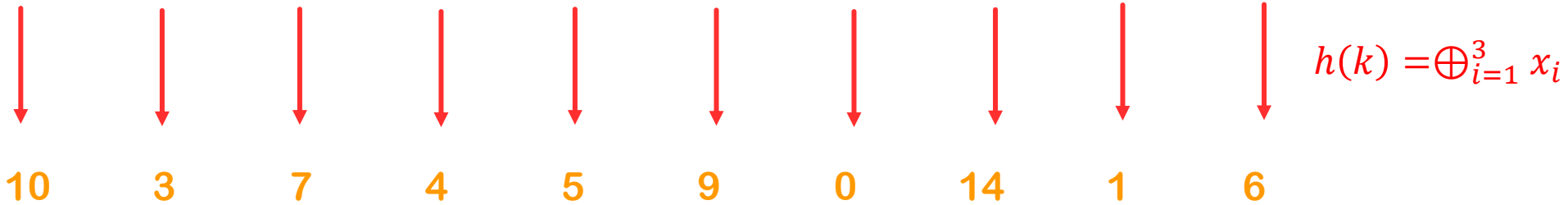
- A slight variation on this theme is to compute an **exclusive-or** of all the components of a key, which could be written mathematically as:

$$h(k) = \bigoplus_{i=1}^d x_i$$

where $\bigoplus$ denotes the bitwise exclusive-or (XOR) operation

EXAMPLE

0	1	2	3	4	5	6	7	8	9
(2,9,1)	(3,8,8)	(4,3,0)	(2,2,4)	(5,3,3)	(1,8,0)	(4,1,5)	(9,1,6)	(2,2,1)	(4,6,4)



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
(4,1,5)	(2,2,1)		(3,8,8)	(2,2,4)	(5,3,3)	(4,6,4)	(4,3,0)		(1,8,0)	(2,9,1)				(9,1,6)

SUMMING COMPONENTS: COLLISION

- Unfortunately, **such hash functions**, which consist of either summing or XORing the components of each key, are actually **not that good** in most applications.
- Such a function is fairly poor at avoiding **collisions** for some cases.
- For example, consider such a hash function for a character string s that **sums the ASCII values** of the characters in s .
- This hash function produces lots of unwanted collisions for some words such as:

"stop", "tops", "pots", "spot"

POLYNOMIAL-EVALUATION FUNCTIONS

- A better hash code in such cases should somehow **take into consideration the positions of the x_i 's**. That is, each index, i , should play a role in the hash function in some way.
- Choose a nonzero constant, $a \neq 1$, and use as a hash function the following:

$$h(k) = x_1 a^{d-1} + x_2 a^{d-2} + \cdots + x_{d-1} a + x_d$$

By Horner's rule

$$h(k) = x_d + a(x_{d-1} + a(x_{d-2} + \cdots + a(x_3 + a(x_2 + ax_1)) \cdots))$$

EXAMPLE

0	1	2	3	4	5	6	7	8	9
(2,9,1)	(3,8,8)	(4,3,0)	(2,2,4)	(5,3,3)	(1,8,0)	(4,1,5)	(9,1,6)	(2,2,1)	(4,6,4)



27



36



22



16



29



20



23



44



13



32

$$h(k) = \sum_{i=1}^3 2^{3-i} x_i$$

... 13 ... 16 ... 20 ... 22 23 ... 27 ... 29 ... 32 ... 36 ... 44

...	(2,2,1)	...	(2,2,4)	...	(1,8,0)	...	(4,3,0)	(4,1,5)	..	(2,9,1)	...	(5,3,3)	...	(4,6,4)	...	(3,8,8)	..	(9,1,6)
-----	---------	-----	---------	-----	---------	-----	---------	---------	----	---------	-----	---------	-----	---------	-----	---------	----	---------

TABULATION-BASED HASH FUNCTIONS

- Assume that all our keys are of the form $k = (x_1, x_2, \dots, x_d)$, for a fixed d , and each x_i is in the range $[0, M - 1]$. Then we can initialize **d tables**, $T_1, T_2, \dots, T_d$, of **size M each**, so that each $T_i[j]$ is a uniformly chosen independent **random number in the range $[0, N - 1]$** . We then can compute the hash function, $h(k)$, as:

$$h(k) = T_1[x_1] \oplus T_2[x_2] \oplus \dots \oplus T_d[x_d]$$

EXAMPLE

0	1	2	3	4	5	6	7	8
(2,4,1)	(3,7,6)	(4,3,0)	(2,2,4)	(5,3,3)	(1,2,0)	(4,1,5)	(5,1,6)	(2,1,6)

	T_1	T_2	T_3
0	9	5	4
1	2	10	0
2	1	4	9
3	4	3	10
4	0	1	6
5	7	8	8
6	5	6	1
7	6	0	5

M (vertical arrow from index 0 to 7)

d (horizontal bracket under indices 0 to 7)

$$n = 9$$

$$M = 8 \Rightarrow [0, M - 1] = [0, 7]$$

$$N = 11 \Rightarrow [0, N - 1] = \{0, 1, 2, 3, \dots, 10\}$$

$$h(2,4,1) = T_1[2] \oplus T_2[4] \oplus T_3[1] = 1 \oplus 1 \oplus 0 = 0 = 0$$

$$h(3,7,6) = T_1[3] \oplus T_2[7] \oplus T_3[6] = 4 \oplus 0 \oplus 1 = 100 \oplus 000 \oplus 001 = 101 = 5$$

HANDLING COLLISIONS AND REHASHING

- There are some methods for handling collisions:

1. Chaining Method

2. Linear Probing

3. Quadratic Probing

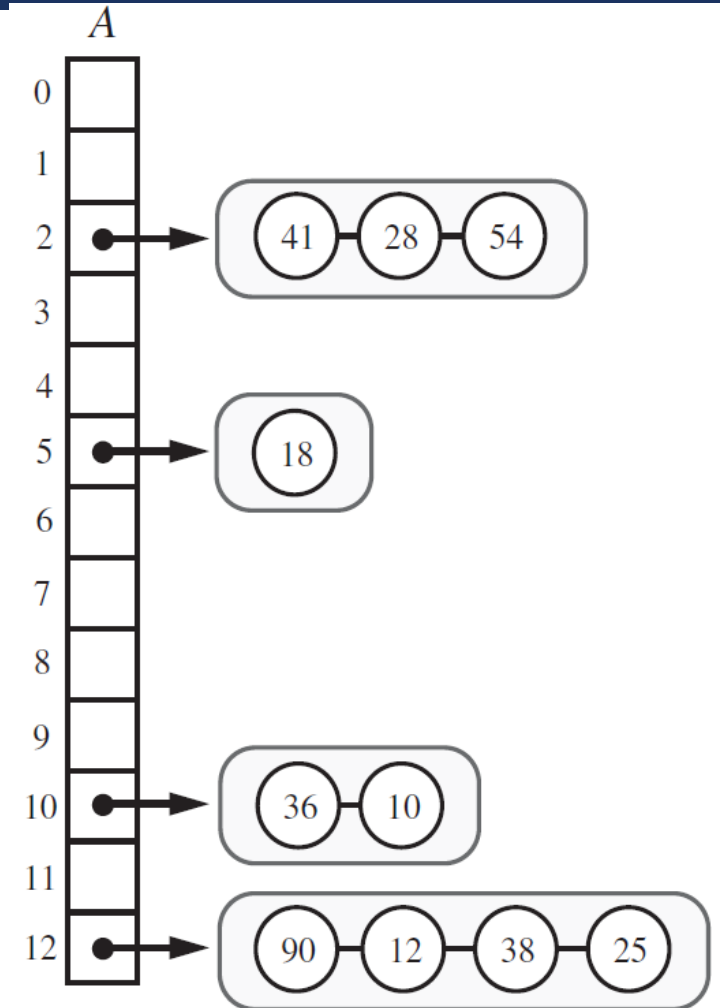
4. Double Hashing

5. Cuckoo Hashing

} Open address

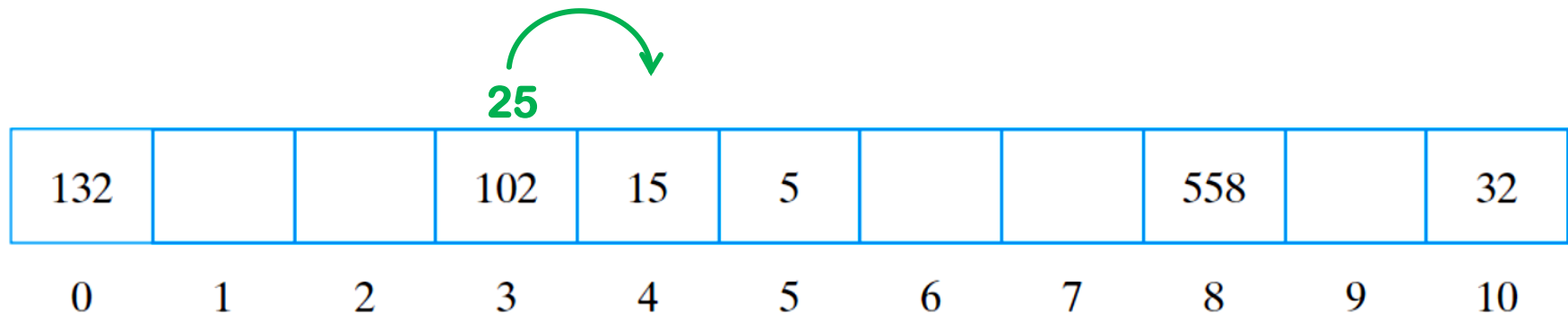
CHAINING METHOD

- Chaining method: A simple and efficient way for dealing with collisions is to have each bucket $A[i]$ store a linked list that holds items (k, v) such that $h(k) = i$.
- Example of a hash table of size 13, storing 10 integer keys, with collisions resolved by the chaining method. The hash function in this case is $h(k) = k \bmod 13$.



LINEAR PROBING

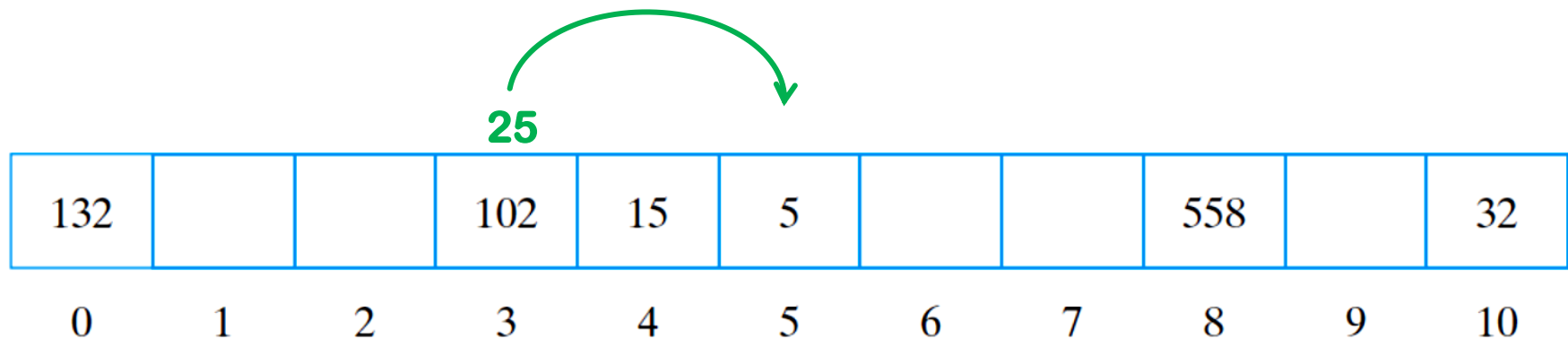
- In this strategy, if we try to insert an item (k, v) into $A[i]$ that is already occupied, where $i = h(k)$, then we try next at $A[(i + 1) \bmod N]$. If it is occupied, then we try $A[(i + 2) \bmod N]$, and so on, until we find an empty cell in A that can accept the new item.



$$h(k) = k \bmod 11$$

LINEAR PROBING

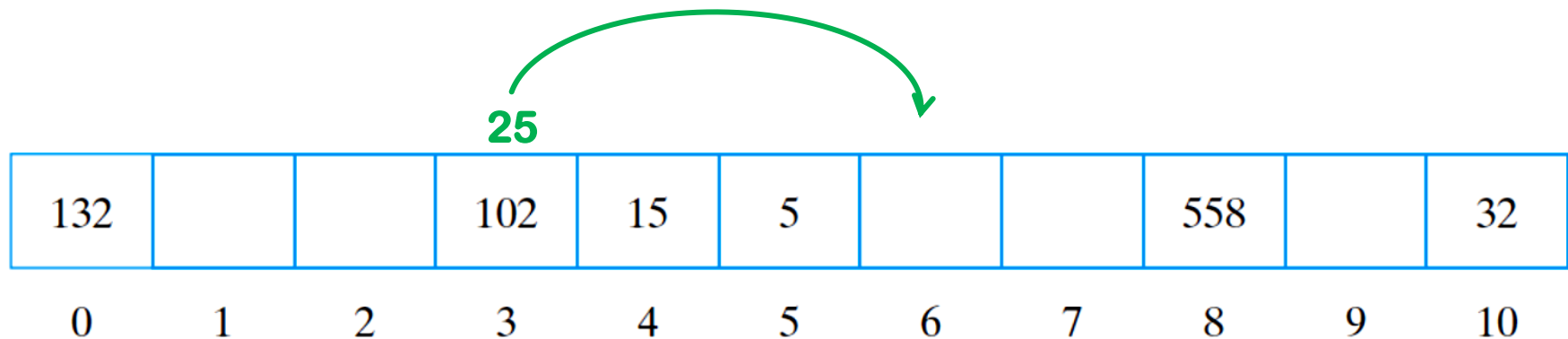
- In this strategy, if we try to insert an item (k, v) into $A[i]$ that is already occupied, where $i = h(k)$, then we try next at $A[(i + 1) \bmod N]$. If it is occupied, then we try $A[(i + 2) \bmod N]$, and so on, until we find an empty cell in A that can accept the new item.



$$h(k) = k \bmod 11$$

LINEAR PROBING

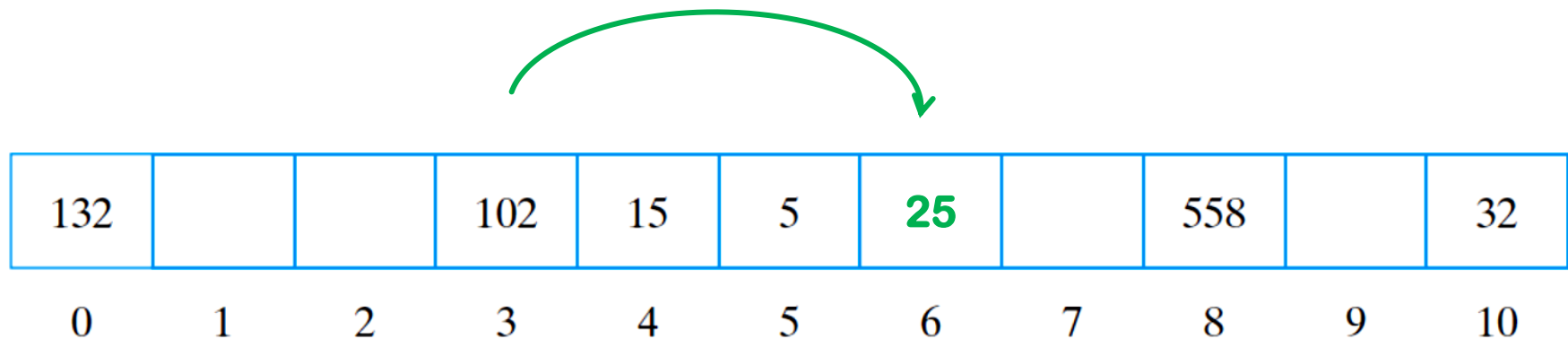
- In this strategy, if we try to insert an item (k, v) into $A[i]$ that is already occupied, where $i = h(k)$, then we try next at $A[(i + 1) \bmod N]$. If it is occupied, then we try $A[(i + 2) \bmod N]$, and so on, until we find an empty cell in A that can accept the new item.



$$h(k) = k \bmod 11$$

LINEAR PROBING

- In this strategy, if we try to insert an item (k, v) into $A[i]$ that is already occupied, where $i = h(k)$, then we try next at $A[(i + 1) \bmod N]$. If it is occupied, then we try $A[(i + 2) \bmod N]$, and so on, until we find an empty cell in A that can accept the new item.



$$h(k) = k \bmod 11$$

QUADRATIC PROBING

- We are trying to insert an item (k, v) into $A[i]$ that is already occupied, where $i = h(k)$: Another strategy, known as *quadratic probing*, involves iteratively trying the cells $A[(i + f(j)) \bmod N]$, for $j = 0, 1, 2, \dots$, where $f(j) = j^2$, until finding an empty cell.

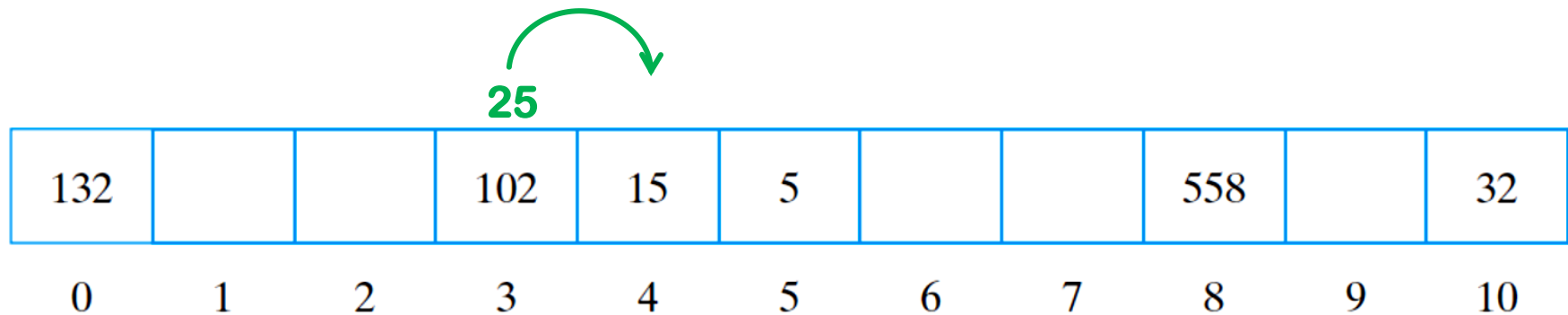
Add(25)

132			102	15	5			558		32
0	1	2	3	4	5	6	7	8	9	10

$$h(k) = k \bmod 11$$

QUADRATIC PROBING

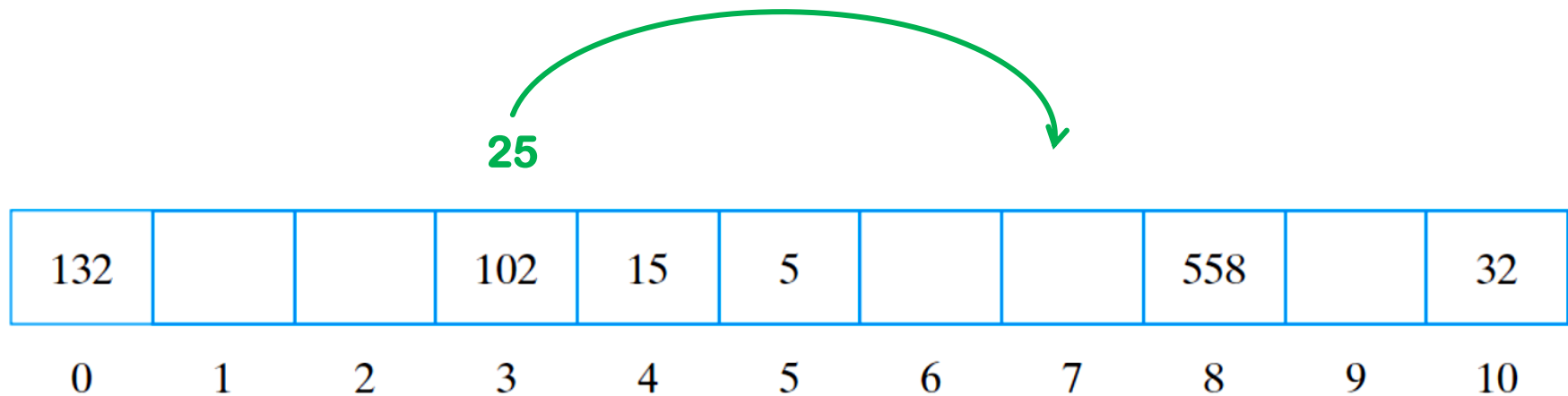
- We are trying to insert an item (k, v) into $A[i]$ that is already occupied, where $i = h(k)$: Another strategy, known as *quadratic probing*, involves iteratively trying the cells $A[(i + f(j)) \bmod N]$, for $j = 0, 1, 2, \dots$, where $f(j) = j^2$, until finding an empty cell.



$$h(k) = k \bmod 11$$

QUADRATIC PROBING

- We are trying to insert an item (k, v) into $A[i]$ that is already occupied, where $i = h(k)$: Another strategy, known as *quadratic probing*, involves iteratively trying the cells $A[(i + f(j)) \bmod N]$, for $j = 0, 1, 2, \dots$, where $f(j) = j^2$, until finding an empty cell.



$$h(k) = k \bmod 11$$

QUADRATIC PROBING

- We are trying to insert an item (k, v) into $A[i]$ that is already occupied, where $i = h(k)$: Another strategy, known as *quadratic probing*, involves iteratively trying the cells $A[(i + f(j)) \bmod N]$, for $j = 0, 1, 2, \dots$, where $f(j) = j^2$, until finding an empty cell.

132			102	15	5		25	558		32
0	1	2	3	4	5	6	7	8	9	10

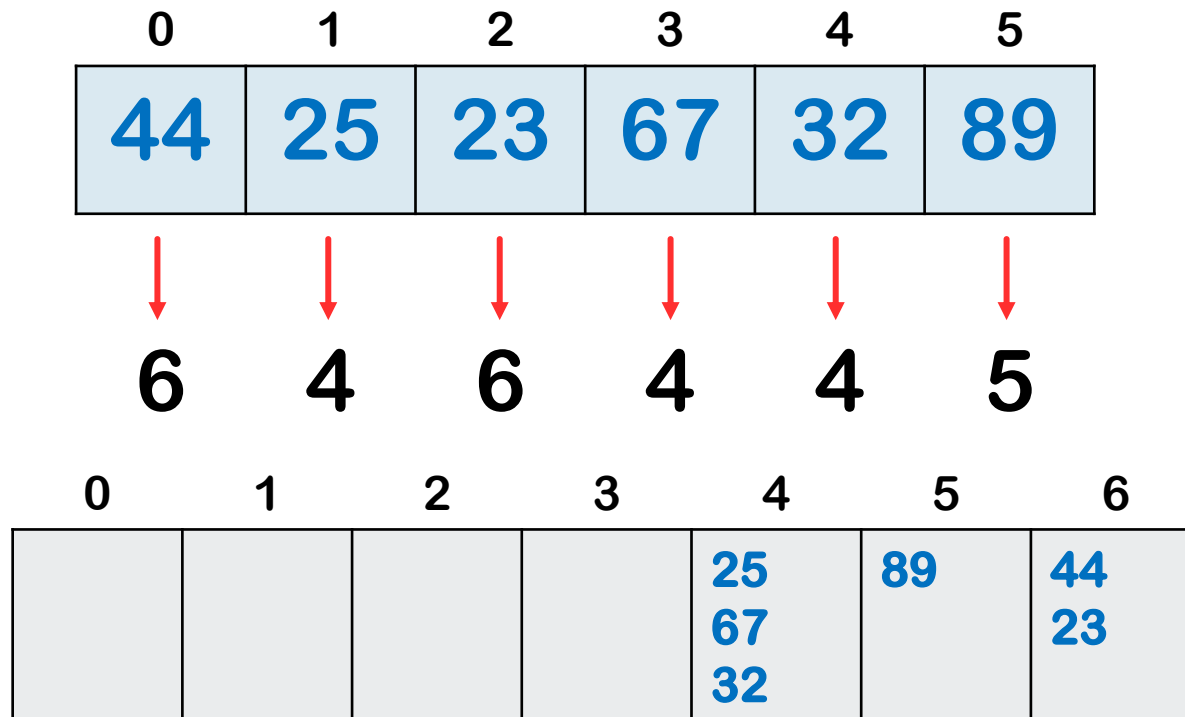
$$h(k) = k \bmod 11$$

DOUBLE HASHING

- In this approach, we choose a secondary hash function, h' , and if h maps some key k to a bucket $A[i]$, with $i = h(k)$, that is already occupied, then we iteratively try the buckets $A[(i + f(j)) \bmod N]$ next, for $j = 1, 2, 3, \dots$, where $f(j) = j \cdot h'(k)$.
- A common choice is $h'(k) = q - (k \bmod q)$, for some prime number $q < N$. Also, N should be a prime.

EXAMPLE

$h(k) = (3k^2 + 2k + 4) \bmod 7$ Random Polynomial Functions



EXAMPLE

$$h'(k) = q - (k \bmod q): q < N$$

$$f(j) = j \cdot h'(k)$$

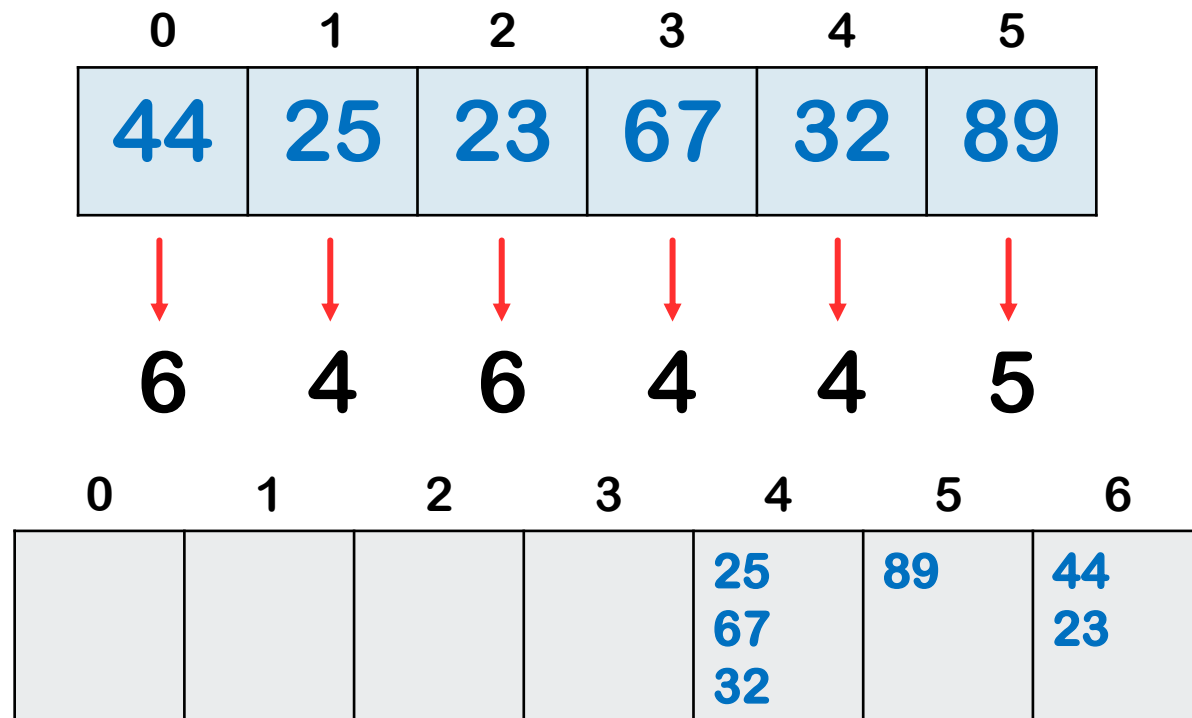
$$A[(i + f(j)) \bmod N]$$

$$N=7, q=3$$

$$h'(k) = 3 - (k \bmod 3)$$

$$f(j) = j \cdot h'(k)$$

$$A[(i + f(j)) \bmod 7]$$



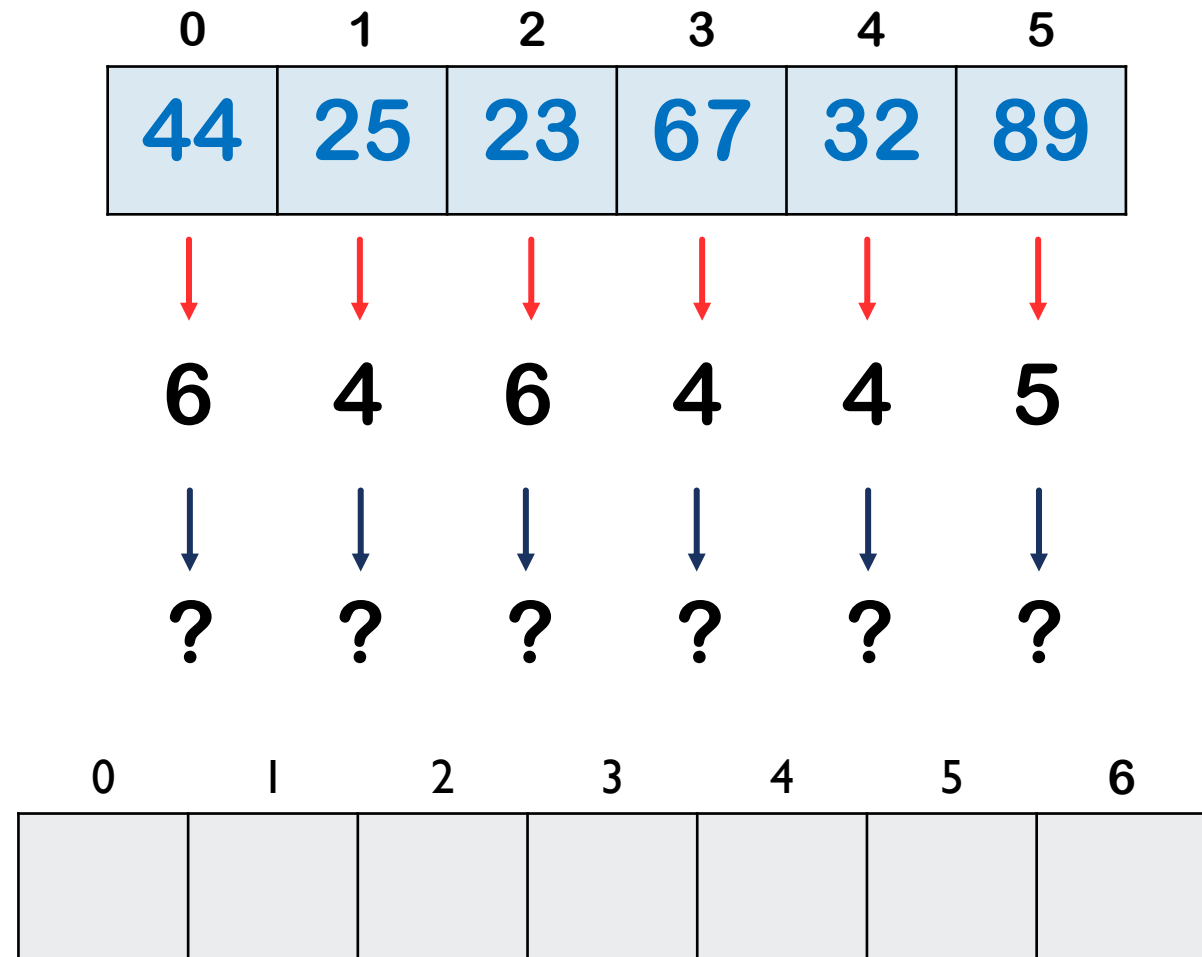
EXAMPLE

$N=7, q=3$

$h'(k) = 3 - (k \bmod 3)$

$f(j) = j \cdot h'(k), j=1,2,\dots$

$A[(i + f(j)) \bmod 7]$



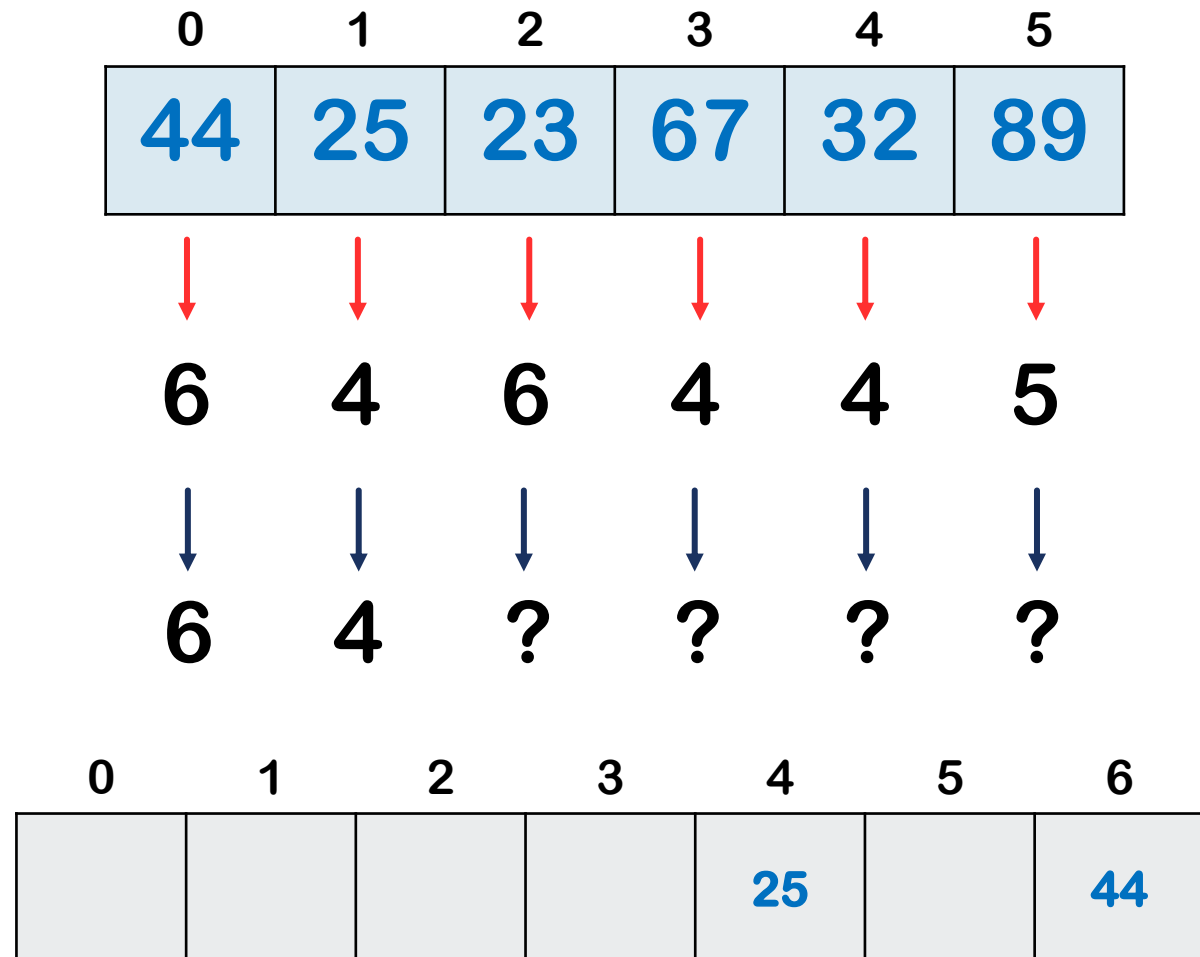
EXAMPLE

$N=7, q=3$

$h'(k) = 3 - (k \bmod 3)$

$f(j) = j \cdot h'(k), j=1,2,\dots$

$A[(i + f(j)) \bmod 7]$



EXAMPLE

$N=7, q=3$

$$h'(k) = 3 - (k \bmod 3)$$

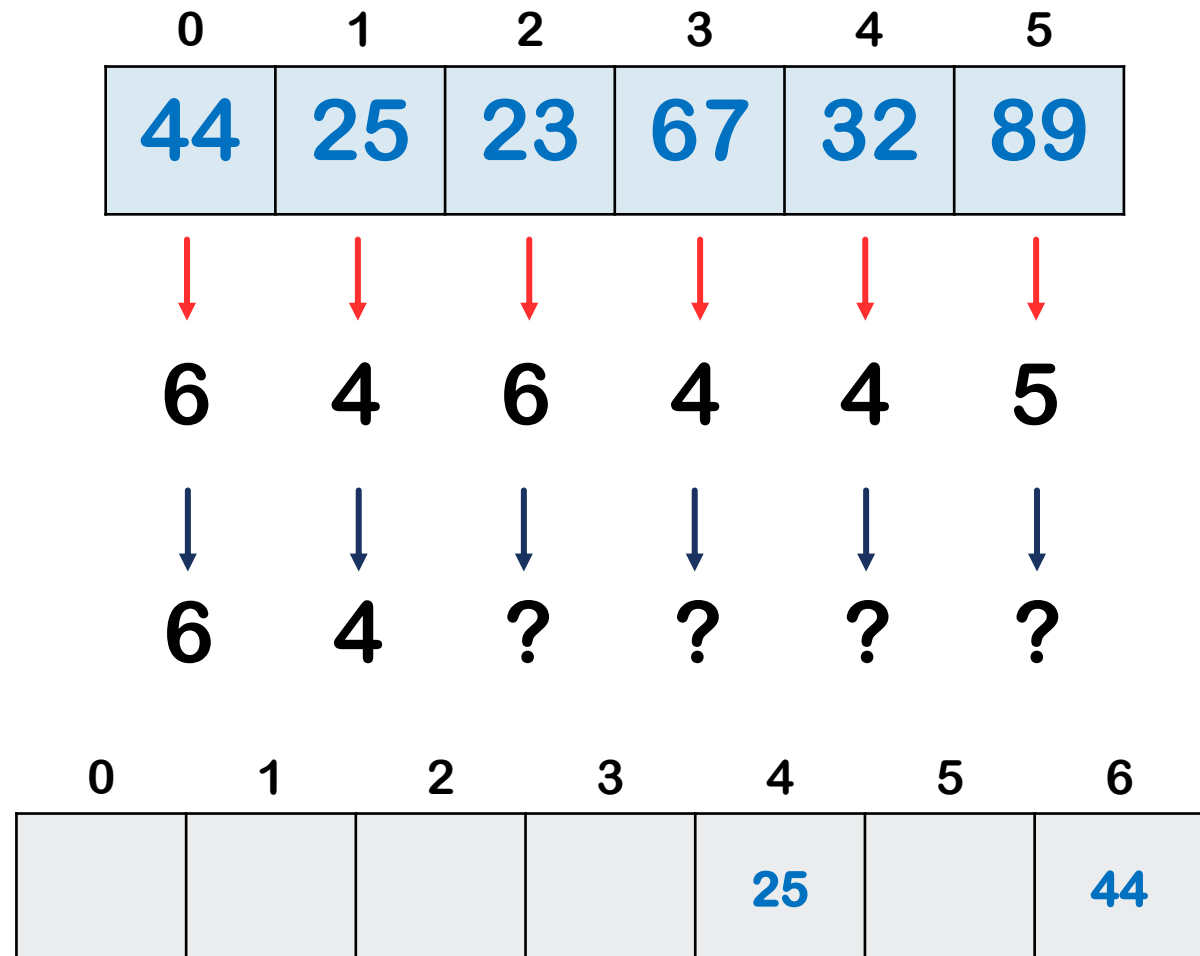
$$f(j) = j \cdot h'(k), j=1,2,\dots$$

$$A[(i + f(j)) \bmod 7]$$

$$h'(23) = 3 - (23 \bmod 3) = 1$$

$$f(1) = 1 \cdot 1 = 1$$

$$(6 + 1) \bmod 7 = 0$$



EXAMPLE

$N=7, q=3$

$$h'(k) = 3 - (k \bmod 3)$$

$$f(j) = j \cdot h'(k), j=1,2,\dots$$

$$A[(i + f(j)) \bmod 7]$$

$$h'(23) = 3 - (23 \bmod 3) = 1$$

$$f(1) = 1 \cdot 1 = 1$$

$$(6 + 1) \bmod 7 = 0$$

0	1	2	3	4	5
44	25	23	67	32	89



6

4

6

4

4

5



6

4

0

?

?

?

0	1	2	3	4	5	6
23				25		44

EXAMPLE

$N=7, q=3$

$$h'(k) = 3 - (k \bmod 3)$$

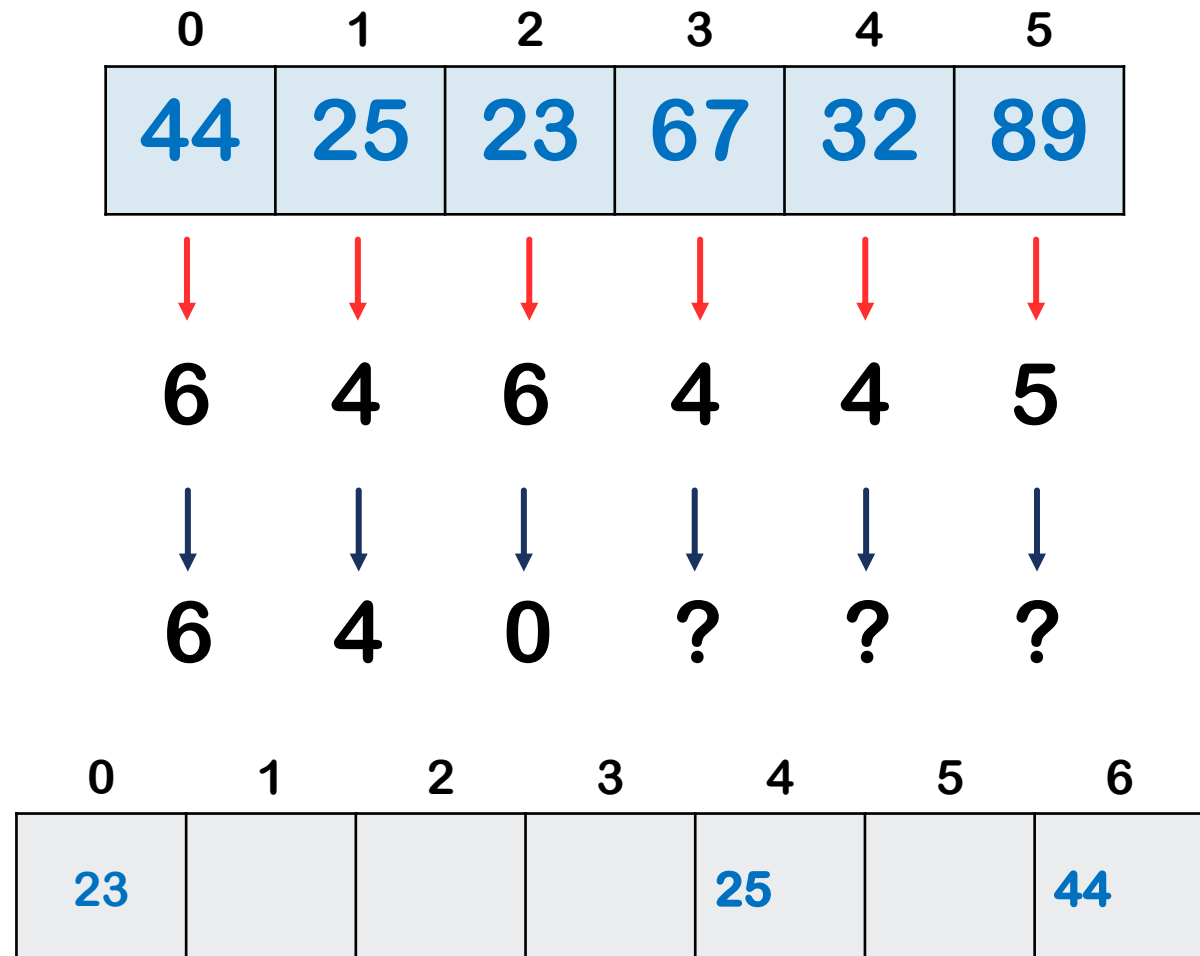
$$f(j) = j \cdot h'(k), j=1,2,\dots$$

$$A[(i + f(j)) \bmod 7]$$

$$h'(67) = 3 - (67 \bmod 3) = 2$$

$$f(1) = 1 \cdot 2 = 2$$

$$(4 + 2) \bmod 7 = 6 \quad \text{X}$$



EXAMPLE

$N=7, q=3$

$$h'(k) = 3 - (k \bmod 3)$$

$$f(j) = j \cdot h'(k), j=1,2,\dots$$

$$A[(i + f(j)) \bmod 7]$$

$$h'(67) = 3 - (67 \bmod 3) = 2$$

$$f(2) = 2 \cdot 2 = 4$$

$$(4 + 4) \bmod 7 = 1$$

0	1	2	3	4	5
44	25	23	67	32	89

↓ ↓ ↓ ↓ ↓ ↓

6	4	6	4	4	5
---	---	---	---	---	---

↓ ↓ ↓ ↓ ↓ ↓

6	4	0	?	?	?
---	---	---	---	---	---

0	1	2	3	4	5	6
23	67			25		44

EXAMPLE

$N=7, q=3$

$h'(k) = 3 - (k \bmod 3)$

$f(j) = j \cdot h'(k), j=1,2,\dots$

$A[(i + f(j)) \bmod 7]$

$h'(32) = 3 - (32 \bmod 3) = 1$

$f(1) = 1 \cdot 1 = 1$

$(4 + 1) \bmod 7 = 5$

0	1	2	3	4	5
44	25	23	67	32	89



6

4

6

4

4

5



6

4

0

?

?

?

0	1	2	3	4	5	6
23	67			25	32	44

CUCKOO HASHING

- **Cuckoo hashing** is a type of **hashing** algorithm used in hash tables to resolve collisions. The idea behind cuckoo hashing is to use **multiple hash functions** and two (or more) hash tables. If a collision occurs, the existing element is "kicked out" and reinserted using another hash function. This process can repeat until all elements are placed or a maximum number of displacements is reached, at which point the hash table may be resized.

EXAMPLE

$$h_1(x) = x \bmod 5$$

$$h_2(x) = (x/5) \bmod 5$$

Input: [10, 20, 30, 25]

- $h_1(10) = 0, h_2(10) = 2$
- $h_1(20) = 0, h_2(20) = 4$
- $h_1(30) = 0, h_2(30) = 1$
- $h_1(25) = 0, h_2(25) = 0$

0	1	2	3	4
0	1	2	3	4
10				
0	1	2	3	4
20				
0	1	2	3	4
30				
0	1	2	3	4
25				

0	1	2	3	4
0	1	2	3	4
0	1	2	3	4
		10		
0	1	2	3	4
		10		20
0	1	2	3	4
	30	10		20

EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$

0	1	2	3	4	5	6

0	1	2	3	4	5	6
	6	11		5		

0	1	2	3	4	5	6
	6	11		2		

0	1	2	3	4	5	6
14	6	11		12		

0	1	2	3	4	5	6	7

0	1	2	3	4	5	6	7

0	1	2	3	4	5	6	7
		5					

0	1	2	3	4	5	6	7
		5			2		

EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

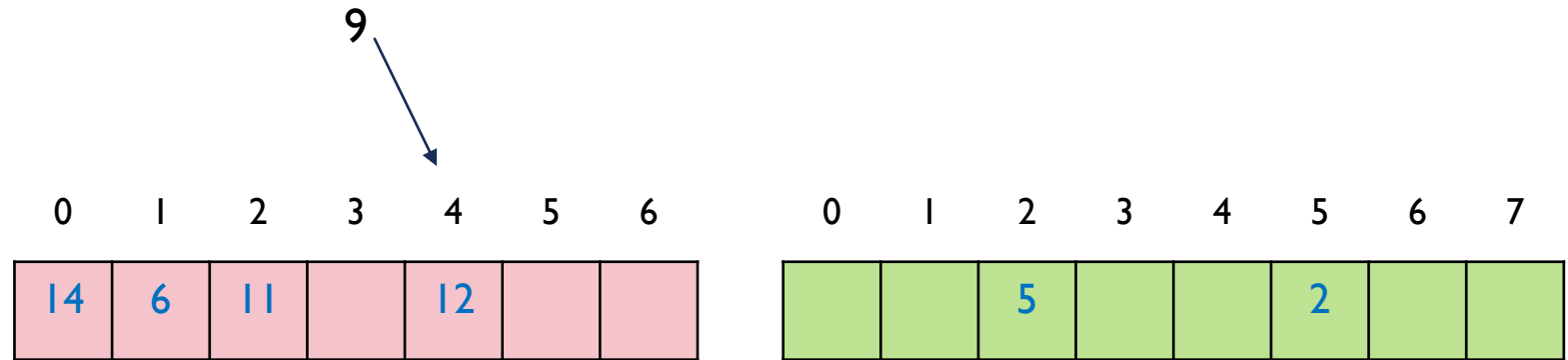
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

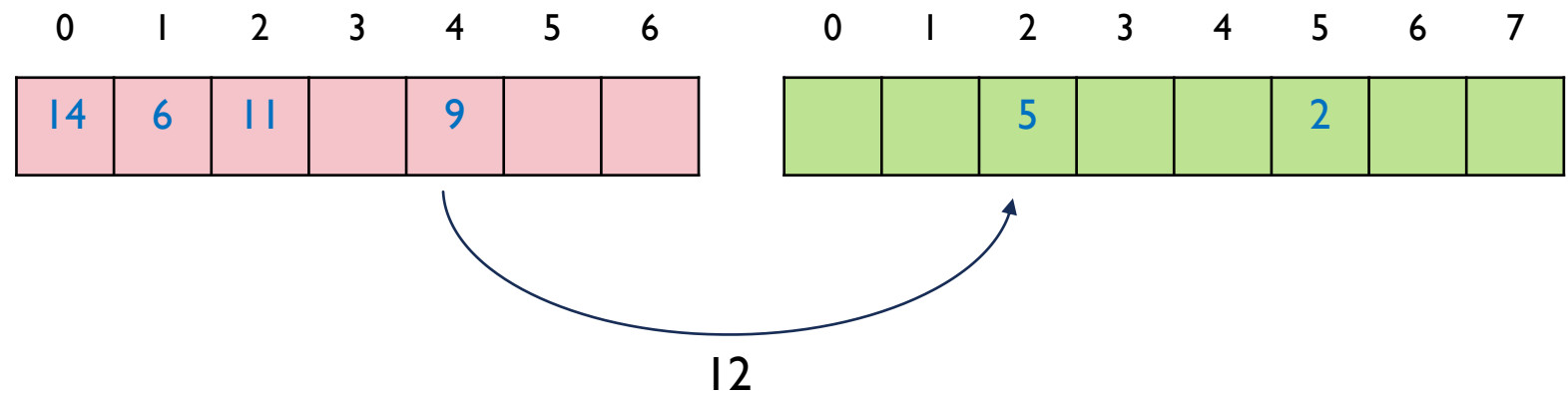
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

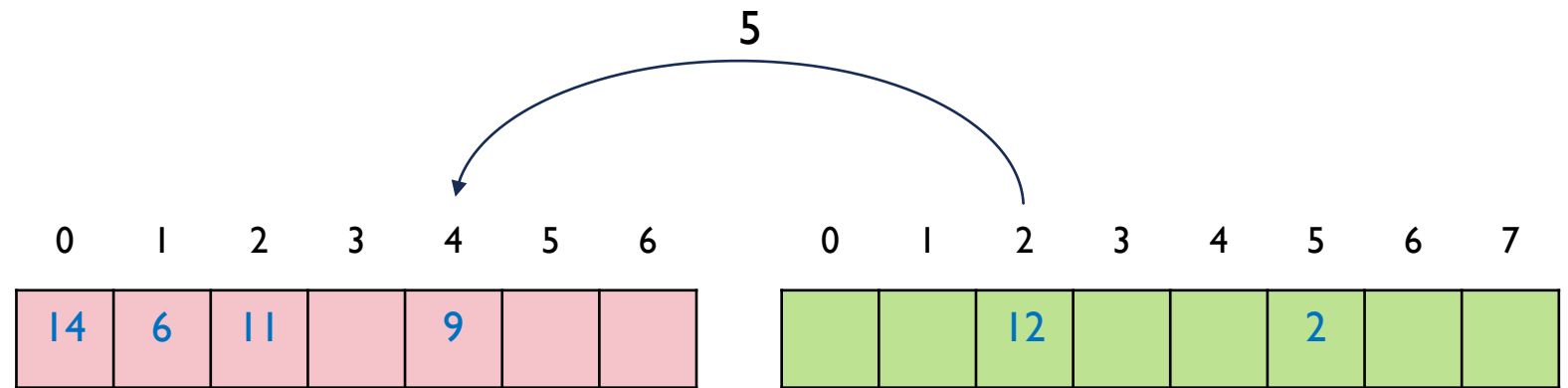
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

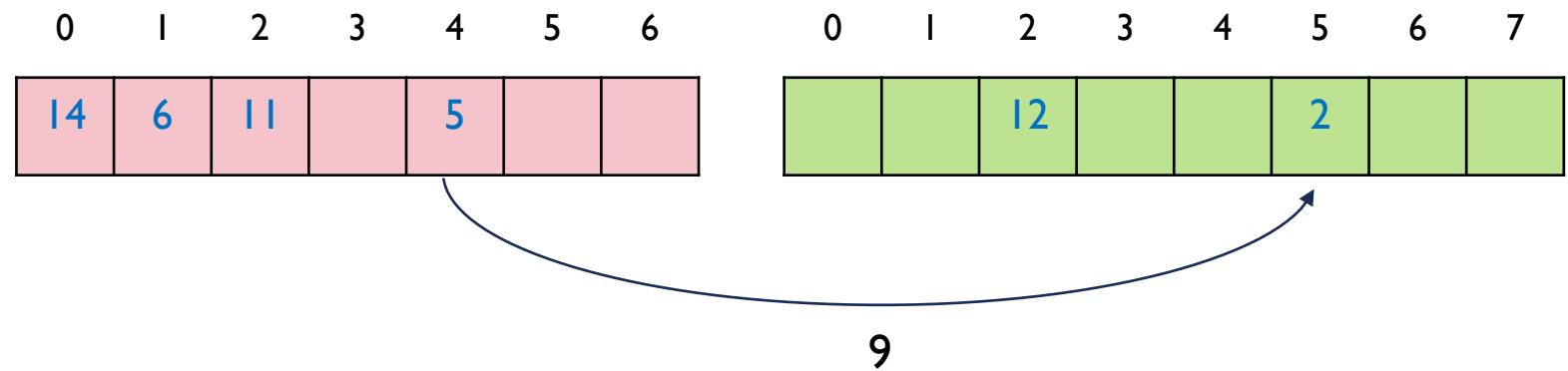
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

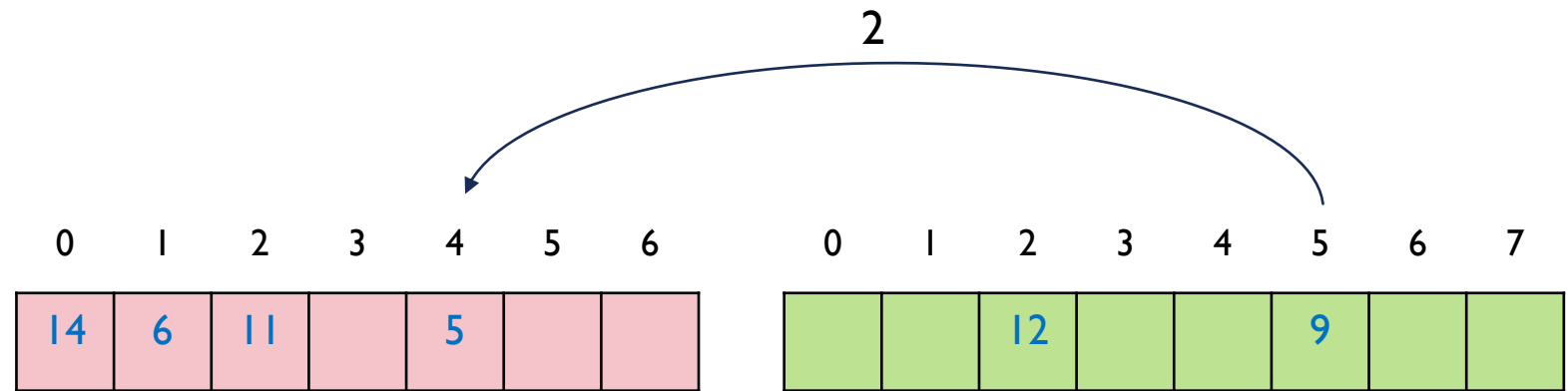
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

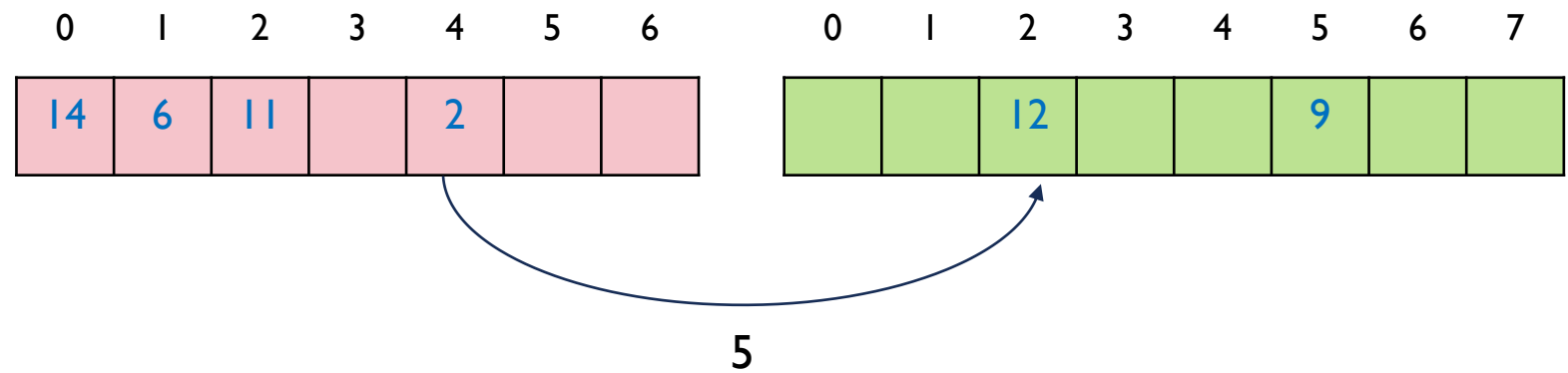
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

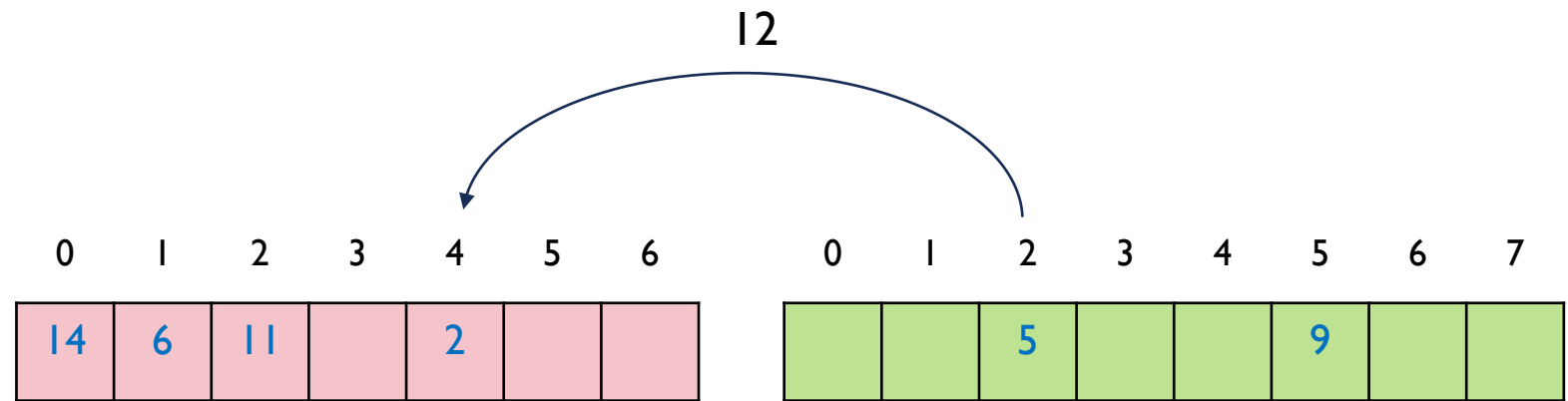
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

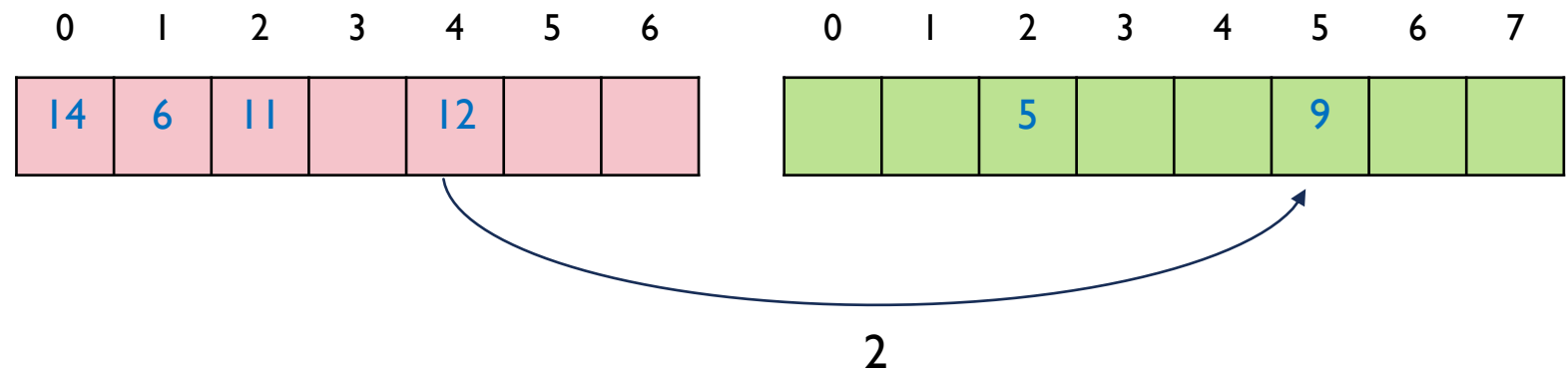
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

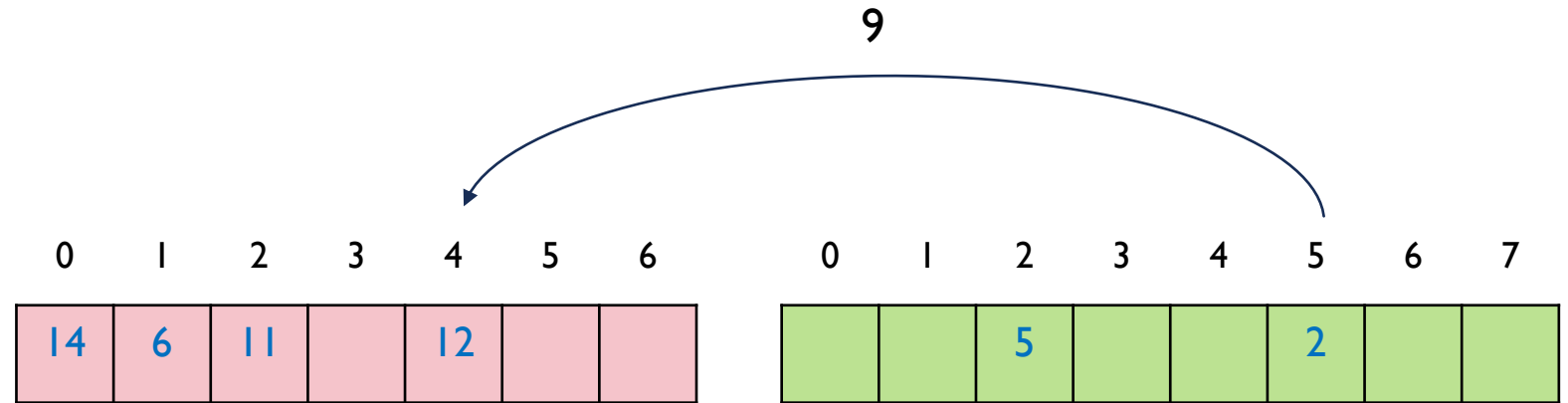
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



EXAMPLE

$$h_1(k) = k^2 \bmod 7$$

$$h_2(k) = 7 - (k \bmod 7)$$

Input: [6,5,11,2,14,12,9,3]

$$h_1(6) = 1, h_2(6) = 1$$

$$h_1(5) = 4, h_2(5) = 2$$

$$h_1(11) = 2, h_2(11) = 3$$

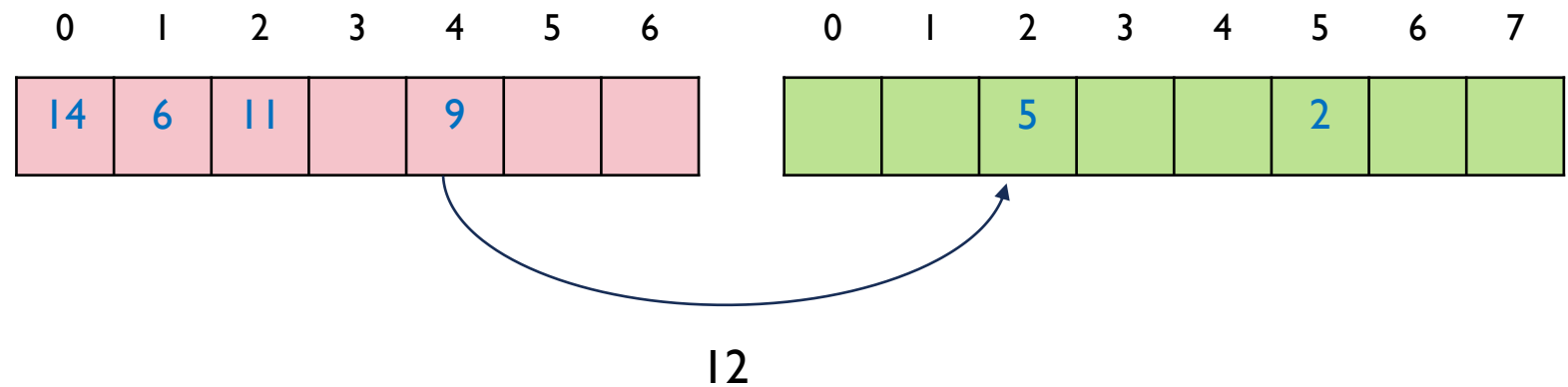
$$h_1(2) = 4, h_2(2) = 5$$

$$h_1(14) = 0, h_2(14) = 7$$

$$h_1(12) = 4, h_2(12) = 2$$

$$h_1(9) = 4, h_2(9) = 5$$

$$h_1(3) = 2, h_2(3) = 4$$



Endless loop: Return to slide 57!!

ANY QUESTION??

HAVE A NICE DAY!